

OfficeStream — Technical Documentation

Version: 1.0.0 **Last Updated:** 25 March 2026 **Author:** OfficeStream Team

1. Project Overview

OfficeStream is a real-time office communication platform inspired by Microsoft Teams. It provides video conferencing, direct messaging, knock-to-call, screen sharing, server-side recording, and office management — all in a modern dark-themed UI.

Key Features

Feature	Description
User Authentication	Register/login with email & password, Google OAuth, JWT access + refresh tokens
Office Management	Create offices, invite members via code, assign roles (owner/admin/member)
Real-time Video Calls	P2P and group calls powered by LiveKit (WebRTC SFU)
Screen Sharing	Share your screen during calls
Server-side Recording	Record meetings via LiveKit Egress — outputs MP4 files
Direct Messaging	Real-time chat between users via Socket.IO
Knock-to-Call	Knock on a user's door to initiate a call
Notifications	Real-time notifications for knocks, invites, messages
Status Management	Online / Away / Busy / Offline status indicators
Swagger API Docs	Interactive API documentation at /api-docs

2. Technology Stack

2.1 Frontend (apps/web/)

Technology	Version	Purpose
Next.js	16.2.1	React framework with App Router (SSR + CSR)
React	19.2.4	UI component library
TypeScript	^5	Type-safe JavaScript
Tailwind CSS	^4	Utility-first CSS framework
@livekit/components-react	^2.9.20	Pre-built LiveKit UI components for video/audio
@livekit/components-styles	^1.2.0	LiveKit component styling
livekit-client	^2.18.0	LiveKit client SDK for WebRTC
axios	^1.13.6	HTTP client with interceptors for API calls
socket.io-client	^4.8.3	Real-time WebSocket client
recordrtc	^5.6.2	Browser-based recording fallback
eslint	^9	Code linting

eslint-config-next	16.2.1	Next.js ESLint rules
@tailwindcss/postcss	^4	PostCSS plugin for Tailwind

2.2 Backend (apps/server/)

Technology	Version	Purpose
Express	^5.2.1	Web framework for REST API
TypeScript	^5.9.3	Type-safe JavaScript
Mongoose	^9.3.1	MongoDB ODM (Object Document Mapper)
Socket.IO	^4.8.3	Real-time bidirectional communication
livekit-server-sdk	^2.15.0	LiveKit server SDK (token generation, Egress control)
jsonwebtoken	^9.0.3	JWT access & refresh token generation
bcryptjs	^3.0.3	Password hashing
passport	^0.7.0	Authentication middleware
passport-google-oauth20	^2.0.0	Google OAuth 2.0 strategy
multer	^2.1.1	File upload handling (recording uploads)
cookie-parser	^1.4.7	Parse cookies (refresh tokens)
cors	^2.8.6	Cross-Origin Resource Sharing
dotenv	^17.3.1	Environment variable management
express-session	^1.19.0	Session management (OAuth state)
nodemailer	^8.0.3	Email sending (password reset)
nanoid	^3.3.11	Unique ID generation (invite codes)
uuid	^13.0.0	UUID generation
swagger-jsdoc	^6.2.8	OpenAPI spec generation
swagger-ui-express	^5.0.1	Swagger UI for API documentation
mongodb-memory-server	^11.0.1	Embedded MongoDB for development
ts-node-dev	^2.0.0	TypeScript dev server with hot reload

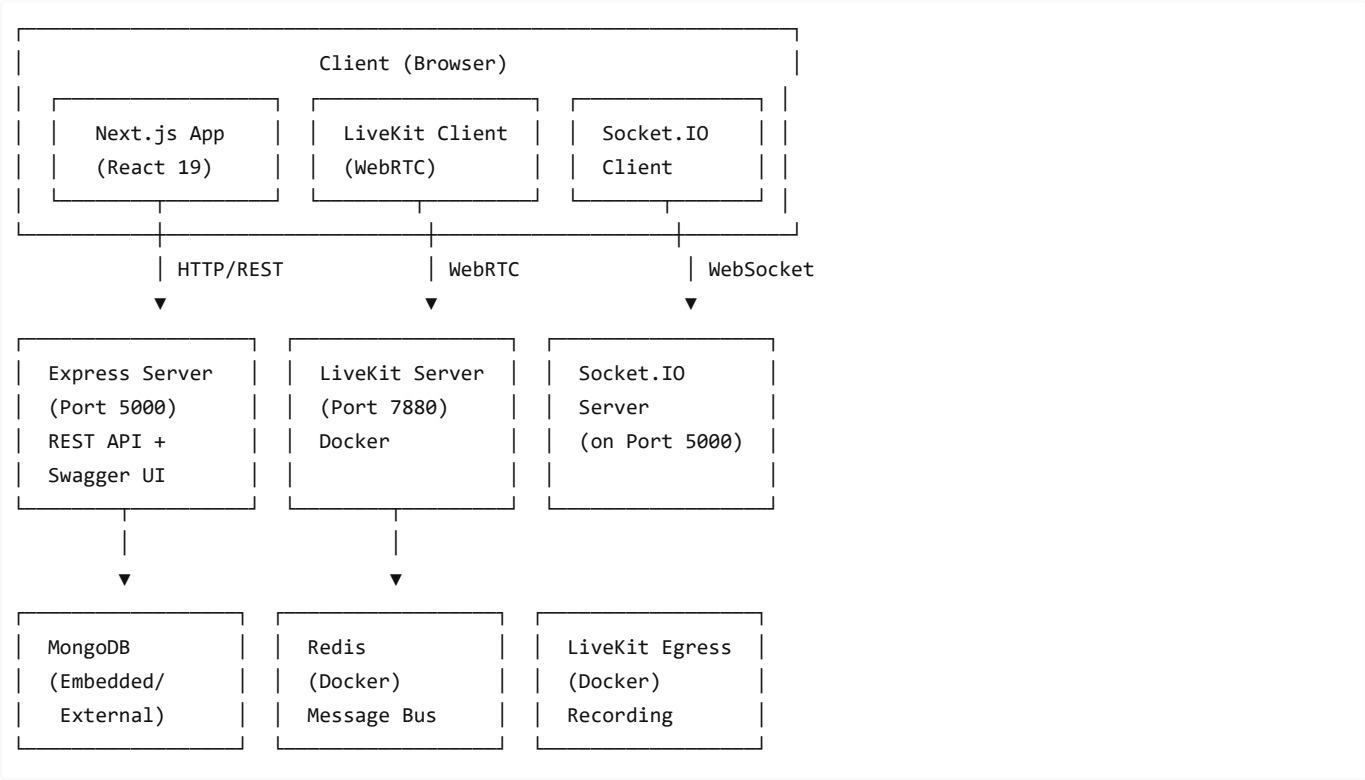
2.3 Infrastructure (Docker)

Service	Image	Purpose
LiveKit Server	livekit/livekit-server:latest	WebRTC SFU — handles all video/audio routing
LiveKit Egress	livekit/egress:latest	Server-side recording & streaming via headless Chrome
Redis	redis:7-alpine	Message bus for LiveKit ↔ Egress communication

2.4 Root / Tooling

Technology	Version	Purpose
concurrently	^9.2.1	Run multiple dev processes in parallel
Docker Compose	v2	Container orchestration for LiveKit stack

3. Architecture Overview



4. Folder Structure

```
officestream/  
├─ package.json           # Root scripts (dev, build, seed)  
├─ docker-compose.yml     # LiveKit + Egress + Redis containers  
├─ .gitignore  
├─  
├─ livekit/  
│   ├── livekit.yaml      # LiveKit server config (ports, keys, Redis)  
│   ├── egress.yaml       # Egress service config (Redis, API keys)  
│   └─ livekit-server.exe  # Local LiveKit binary (gitignored)  
├─  
├─ apps/  
│   ├── server/           # == BACKEND (Express + TypeScript) ==  
│   │   ├── package.json  
│   │   ├── tsconfig.json  
│   │   ├── .env           # Environment variables (gitignored)  
│   │   ├── .env.example   # Template for .env  
│   │   ├── recordings/    # Recorded MP4 files (gitignored)  
│   │   └─ src/  
│   │       └─ index.ts    # Entry point – starts HTTP + Socket.IO
```

```

├── app.ts          # Express setup, middleware, route mounting
├── config/
│   └── db.ts       # Re-exports connectDB from lib/db.ts
├── lib/
│   ├── constants.ts # All magic strings, durations, enums
│   ├── db.ts        # MongoDB connection (auto-embedded in dev)
│   ├── livekit.ts   # LiveKit token generation, EgressClient
│   ├── mailer.ts    # Email sending via Nodemailer
│   ├── passport.ts  # Google OAuth Passport config
│   └── swagger.ts   # OpenAPI/Swagger spec definition
├── middleware/
│   └── auth.ts      # requireAuth JWT middleware
├── models/
│   ├── User.ts      # User schema (name, email, password, status, role)
│   ├── Office.ts    # Office schema (name, owner, members, invite codes)
│   ├── Message.ts   # DM message schema (from, to, content)
│   ├── Notification.ts # Notification schema (type, message, read)
│   └── Recording.ts # Recording schema (egressId, status, filename)
├── controllers/
│   ├── authController.ts      # Register, login, refresh, logout, me
│   ├── officeController.ts    # Create, join, invite, manage members
│   ├── messageController.ts   # Get conversations, unread counts
│   ├── notificationController.ts # List, read, delete notifications
│   ├── recordingController.ts # Start/stop egress, upload, download
│   └── tokenController.ts     # LiveKit token generation
├── routes/
│   ├── auth.ts      # /api/auth/*
│   ├── authGoogle.ts # /api/auth/google/*
│   ├── offices.ts   # /api/offices/*
│   ├── messages.ts  # /api/messages/*
│   ├── notifications.ts # /api/notifications/*
│   ├── recording.ts  # /api/recording/*
│   └── token.ts      # /api/token/*
├── socket/
│   └── index.ts      # Socket.IO initialization, events, helpers
├── scripts/
│   └── seed.ts       # Seeds admin user + default office
└── web/              # == FRONTEND (Next.js + TypeScript) ==
    ├── package.json
    ├── tsconfig.json
    ├── next.config.ts
    ├── postcss.config.mjs
    ├── .env.local    # Frontend env vars (gitignored)
    │
    ├── app/
    │   ├── layout.tsx # Root layout (AuthProvider + SocketProvider)
    │   └── page.tsx   # Landing - redirects to /dashboard or /login

```

```

├── globals.css      # Global styles + Tailwind imports
├── login/page.tsx   # Login page (with Dev Login button)
├── signup/page.tsx  # Registration page
├── auth/callback/page.tsx # Google OAuth callback handler
├── dashboard/
│   ├── layout.tsx   # Auth guard wrapper
│   └── page.tsx     # Main dashboard (office floor, video, chat)
├── room/[roomId]/page.tsx # Conference room page
├── call/[callId]/page.tsx # P2P call page
└── stream/[streamId]/page.tsx # Live streaming page

├── components/
│   ├── TopBar.tsx      # Top navigation bar
│   ├── Sidebar.tsx     # Left sidebar (members, rooms)
│   ├── VideoGrid.tsx   # Video tile layout
│   ├── ControlBar.tsx  # Call controls (mute, camera, record, etc.)
│   ├── ChatPanel.tsx   # DM chat panel
│   ├── OfficeSetup.tsx # Create/join office wizard
│   ├── KnockOverlay.tsx # Incoming/outgoing knock UI
│   ├── InviteModal.tsx # Invite members modal
│   ├── NotificationDropdown.tsx # Notification bell dropdown
│   └── StatusDot.tsx   # Online/away/busy status indicator
├── context/
│   ├── AuthContext.tsx # Auth state, login/logout, token refresh
│   └── SocketContext.tsx # Socket.IO connection management
├── hooks/
│   ├── useRecording.ts # Recording start/stop logic
│   ├── useScreenShare.ts # Screen sharing hook
│   ├── useCallTimer.ts # Call duration timer
│   └── useVideoGrid.ts # Video grid layout calculations
└── lib/
    └── api.ts          # Axios instance with refresh token interceptor

```

5. Why These Technologies?

5.1 Why MongoDB?

- **Flexible Schema:** Users, offices, messages, and recordings have varying structures — MongoDB's document model handles this naturally without rigid migrations.
- **Embedded Dev Mode:** Using `mongodb-memory-server`, developers don't need to install MongoDB separately. The app auto-starts an embedded instance with persistent storage in `.data/mongodb`.
- **Mongoose ODM:** Provides schema validation, middleware, and TypeScript support on top of MongoDB.
- **Real-time Friendly:** MongoDB Change Streams (future) can power real-time data sync alongside Socket.IO.

5.2 Why LiveKit?

- **Open-Source WebRTC SFU:** LiveKit is a production-grade Selective Forwarding Unit — it routes media streams efficiently without mixing, keeping latency low.
- **Built-in Egress:** LiveKit Egress provides server-side recording without any third-party service. It runs a headless Chrome that composites all participants into a single MP4.
- **Scalable:** LiveKit handles hundreds of participants per room and can scale horizontally with Redis.
- **SDK Support:** Official SDKs for both server (token generation, room management, egress control) and client (React components, track management).

5.3 Why Docker?

- **LiveKit Egress Requires It:** The Egress service runs headless Chrome inside a Linux container — Docker is the only way to run it on Windows.
- **Consistent Environment:** Redis, LiveKit Server, and Egress all run in isolated containers with the same config on every developer's machine.
- **One Command Setup:** `docker compose up -d` starts the entire LiveKit infrastructure.
- **Volume Mounts:** Recorded files are written to `/out` inside the Egress container, which maps to `apps/server/recordings/` on the host via Docker volumes.

5.4 Why Redis?

- **LiveKit ↔ Egress Communication:** Redis acts as the message bus (pub/sub) between the LiveKit server and the Egress service. When you call `startRoomCompositeEgress()`, the request goes through Redis to the Egress worker.
- **Required by LiveKit:** Without Redis, the Egress service cannot receive recording commands from the LiveKit server.

5.5 Why Socket.IO?

- **Real-time Events:** Knock-to-call, presence updates, DM messages, and notifications all need instant delivery.
- **Fallback Transport:** Socket.IO automatically falls back from WebSocket to HTTP long-polling if WebSocket is blocked.
- **Room-based Broadcasting:** Socket.IO rooms map perfectly to offices and user channels.

5.6 Why JWT (Access + Refresh Tokens)?

- **Stateless Auth:** The server doesn't store sessions — the JWT contains the user ID and expiry.
- **Short-lived Access Token (15 min):** Limits exposure if a token is intercepted.
- **Long-lived Refresh Token (7 days):** Stored as an `httpOnly` cookie — JavaScript can't access it, preventing XSS theft.
- **Max 5 Sessions:** The server limits concurrent refresh tokens per user to 5.

6. API Documentation (Swagger)

6.1 Accessing Swagger UI

Open your browser and navigate to:

```
http://localhost:5000/api-docs
```

The raw OpenAPI JSON spec is available at:

```
http://localhost:5000/api-docs.json
```

6.2 How to Authorize in Swagger

Protected endpoints require a JWT Bearer token. Follow these steps:

Step 1: Get an Access Token

1. In Swagger UI, expand the **Auth** section
2. Click on **POST /api/auth/login**
3. Click **"Try it out"**
4. Enter the request body:

```
{
  "email": "admin@officestream.com",
  "password": "admin123"
}
```

5. Click **"Execute"**

6. From the response, copy the `accessToken` value (without quotes)

Step 2: Authorize

1. Scroll to the **top** of the Swagger page
2. Click the green **"Authorize"** button (lock icon)
3. In the popup, paste your access token into the **Value** field
4. Click **"Authorize"**, then **"Close"**

Step 3: Use Protected Endpoints

All endpoints with a lock icon () will now include your token automatically. You can test any endpoint by clicking "Try it out" → filling parameters → "Execute".

Note: The access token expires after 15 minutes. If you get a 401 error, repeat the login and authorize steps.

6.3 API Endpoints Summary

Category	Endpoints	Auth Required
Health	GET /health	No
Auth	8 endpoints (register, login, refresh, logout, me, status, Google OAuth)	Partial
Offices	6 endpoints (create, get, join, invite, add member, update role)	Yes
Notifications	4 endpoints (list, read all, read one, delete)	Yes
Token	2 endpoints (get LiveKit token, guest token)	Partial
Messages	2 endpoints (unread counts, conversation)	Yes
Recording	7 endpoints (start, stop, upload, status, list, download, delete)	Yes
Total	29 endpoints	

7. Recording System — How It Works

7.1 Architecture

```
User clicks "Record"
  |
  ▼
POST /api/recording/start { roomName }
  |
  ▼
Express Server → EgressClient.startRoomCompositeEgress()
  |
  ▼ (via Redis pub/sub)
LiveKit Egress Service
  |
  ▼
Headless Chrome joins the LiveKit room
Composites all video/audio into one stream
Encodes to MP4 → writes to /out (Docker volume)
  |
  ▼
/out maps to → apps/server/recordings/ (host filesystem)
```

7.2 Recording Lifecycle

Step	Status	What Happens
1	recording	Egress starts, Chrome joins room, encoding begins
2	processing	User clicks stop, egress finalizes the MP4 file
3	ready	File is available for download
4	failed	Egress error or timeout

7.3 File Storage

- **Location:** apps/server/recordings/
- **Format:** MP4 (H.264 + AAC)
- **Naming:** {roomName}-{timestamp}.mp4
- **Download:** GET /api/recording/{id}/download

8. Getting Started

8.1 Prerequisites

- **Node.js** v20+
- **Docker Desktop** (for LiveKit + Egress + Redis)
- **Git**

8.2 Installation

```
# Clone the repository
git clone <repo-url>
cd officestream

# Install dependencies
npm install
cd apps/server && npm install && cd ../..
cd apps/web && npm install && cd ../..
```

8.3 Environment Variables

Copy the example env file and customize:

```
cp apps/server/.env.example apps/server/.env
```

Key variables:

Variable	Default	Description
PORT	5000	Express server port
MONGODB_URI	mongodb://localhost:27017/officestream	MongoDB connection (auto-embedded in dev)
JWT_ACCESS_SECRET	change_me	Secret for access tokens
JWT_REFRESH_SECRET	change_me	Secret for refresh tokens
CLIENT_ORIGIN	http://localhost:3000	Frontend URL (CORS)

LIVEKIT_URL	ws://localhost:7880	LiveKit server WebSocket URL
LIVEKIT_API_KEY	devkey	LiveKit API key (must match livekit.yaml)
LIVEKIT_API_SECRET	secret	LiveKit API secret (must match livekit.yaml)

Frontend (apps/web/.env.local):

```
NEXT_PUBLIC_API_URL=http://localhost:5000
NEXT_PUBLIC_NODE_ENV=development
```

8.4 Running the App

Single command to start everything:

```
npm run dev
```

This starts:

- 1. **Docker Compose** — Redis + LiveKit Server + LiveKit Egress
- 2. **Express Server** — Backend API on port 5000
- 3. **Next.js** — Frontend on port 3000

8.5 Seed Data

```
npm run seed
```

Creates a default admin user and office:

- **Email:** admin@officestream.com
- **Password:** admin123
- **Office:** OfficeStream HQ

9. Design System

The UI follows a Microsoft Teams-inspired dark theme:

Element	Color	Hex
Sidebar	Dark gray	#252526
Top Bar	Medium gray	#2d2d2d
Main background	Near-black	#1a1a1a
Accent (primary)	Teams purple	#6264a7
Accent (hover)	Light purple	#7b83c7
Online	Green	#92c353
Away	Yellow	#f8d22a
Busy	Red	#c4314b
Offline	Gray	#5a5a5a

Border	Subtle gray	#3a3a3a
Text (primary)	Light gray	#d4d4d4
Text (muted)	Medium gray	#9d9d9d

10. Use Cases

Use Case 1: Office Setup

1. Admin registers → creates an office
2. Generates invite code → shares with team
3. Members register → join with invite code
4. Everyone appears on the office floor

Use Case 2: Video Call

1. User A knocks on User B
2. User B accepts → both join a LiveKit room
3. Video/audio streams via WebRTC SFU
4. Screen sharing available via control bar

Use Case 3: Meeting Recording

1. During a call, any participant clicks "Record"
2. Server starts LiveKit Egress → headless Chrome joins the room
3. All audio/video is composited into a single MP4
4. When stopped, the file is saved and the user gets a DM notification
5. Recording is downloadable from the recordings list

Use Case 4: Direct Messaging

1. Click on a user in the sidebar
2. Chat panel opens → real-time messaging via Socket.IO
3. Unread counts shown as badges
4. Messages persist in MongoDB

11. Feature Flows — Components, Methods, APIs & Storage

This section traces the exact flow for each feature: which frontend component triggers it, what method/function is called, which API endpoint is hit, what backend controller handles it, and which MongoDB collection stores the data.

11.1 Master Flow Table

Feature	Frontend Component	Frontend Method	API / Socket Event	Backend Controller
Register	signup/page.tsx	handleSubmit()	POST /api/auth/register	authController.register
Login	login/page.tsx	doLogin(email, pw)	POST /api/auth/login	authController.login
Token Refresh	lib/api.ts (interceptor)	Auto on 401	POST /api/auth/refresh	authController.refresh
Logout	AuthContext.tsx	logout()	POST /api/auth/logout	authController.logout
Update Status	Sidebar.tsx	setStatus()	PATCH /api/auth/me/status + emit set_status	authController.updateStatus + socket


```

└─ user.save() [users collection]
└─ res.cookie('refreshToken', ...) [httpOnly cookie]
    res.json({ accessToken, user })

```

Storage:

- users collection → email, password (bcrypt), refreshTokens[]
- Client localStorage → accessToken (JWT, 15min)
- Browser cookie → refreshToken (httpOnly, 7 days, path=/api/auth)

11.3 Chat / Direct Messaging Flow (Detailed)

FRONTEND

ChatPanel.tsx

```

└─ On Open: api.get('/api/messages/{userId}') → load history
└─ send(): socket.emit('dm_send', {toId, content})
└─ Listen: socket.on('dm_message', handler) → show message

```

Socket.IO



BACKEND

socket/index.ts → on 'dm_send'

```

└─ Message.create({ from, to, content }) [messages collection]
└─ io.to(userSocketRoom(toId)).emit('dm_message', payload)
└─ io.to(userSocketRoom(fromId)).emit('dm_message', payload)

```

routes/messages.ts → GET /api/messages/:userId

```

└─ messageController.getConversation(req, res)
└─ Message.find({from:to}).sort({createdAt:1}).limit(100)
└─ Message.updateMany({read:false → true}) [mark read]

```

Storage:

- messages collection → from, to, content, type, read, createdAt
- Index: { from: 1, to: 1 } (compound index for fast lookups)

11.4 Knock / Video Call Flow (Detailed)

User A (Caller)

User B (Receiver)

dashboard/page.tsx

```

handleKnock(member)
└─ roomId = randomUUID()
└─ emit('knock',{toId,roomId})
└─ shows OutgoingKnockOverlay

```

socket.on('call_accepted')

└─ router.push('/room/{roomId}')

dashboard/page.tsx

```

socket.on('incoming_call')
└─ shows IncomingKnockOverlay

```

handleAcceptCall()

```

└─ emit('call_accept',
    {toId, roomId})

```

└─ router.push('/room/...')

Both users join the same LiveKit room via WebRTC

```
Token: GET /api/token?roomName={roomId}
```

Socket Events:

- 'knock' → server relays as 'incoming_call'
- 'call_accept' → server relays as 'call_accepted'
- 'call_decline' → server relays as 'call_declined'

Storage: No database write – calls are ephemeral (LiveKit manages rooms)

11.5 Recording Flow (Detailed)

FRONTEND

```
useRecording.ts hook (used in ControlBar.tsx)
├─ start(): api.post('/api/recording/start', {roomName})
│   └─ stores egressId in ref
│       └─ emit('recording:started', {roomName})
└─ stop(): api.post('/api/recording/stop', {egressId})
    └─ emit('recording:stopped', {roomName})
```

HTTP POST



BACKEND

```
recordingController.startRecording()
├─ Recording.findOne({roomName, status:'recording'}) [check]
├─ egressClient.startRoomCompositeEgress(roomName, output)
│   └─ EncodedFileOutput: MP4, path: /out/{name}-{ts}.mp4
│       └─ Goes via Redis → LiveKit Egress container
├─ Recording.create({...status:'recording'})
└─ res.json({ recordingId, egressId })

recordingController.stopRecording()
├─ egressClient.stopEgress(egressId)
├─ Recording.update({status:'processing'})
└─ pollEgressCompletion() [background, every 2s, max 5min]
    └─ On EGRESS_COMPLETE:
        ├── Recording.update({status:'ready', size, duration})
        ├── Message.create({content:'Recording ready...'})
        └─ io.emit('dm_message') → notify user
    └─ On EGRESS_FAILED:
        └─ Recording.update({status:'failed', error})
```

Storage:

- recordings collection → roomName, egressId, filename, status, size, duration
- File system → apps/server/recordings/{roomName}-{timestamp}.mp4
- messages collection → "Recording ready" DM notification

11.6 Token Refresh Flow (Detailed)

FRONTEND (lib/api.ts – Axios Response Interceptor)

Any API call returns 401?

- └ api.post('/api/auth/refresh', {}, {withCredentials:true})
 - └ Browser auto-sends httpOnly refreshToken cookie
 - └ On success: save new accessToken, retry original req
 - └ On failure: dispatch 'auth:logout' event → clear state



BACKEND (authController.refresh)

1. Read refreshToken from cookie
2. jwt.verify(token, JWT_REFRESH_SECRET)
3. User.findById(userId)
4. Check: user.refreshTokens.includes(token)?
 - └ If NO → token reuse detected! Clear ALL tokens (breach)
5. Rotate tokens:
 - └ Remove old token from array
 - └ Generate new refresh token
 - └ Push new token (max 5 sessions, oldest removed)
 - └ user.save()
6. Set new httpOnly cookie + return new accessToken

Security:

- Refresh token rotation (old token invalidated after each use)
- Reuse detection (if stolen token is used → all sessions killed)
- Max 5 concurrent sessions per user

11.7 Presence / Status Flow

User connects via Socket.IO

- Server: User.update({status:'online'})
- Broadcast: emit('presence', {userId, status:'online'}) to office room

User sets status (e.g. "busy")

- Client: emit('set_status', 'busy')
- Server: User.update({status:'busy'})
- Broadcast: emit('presence', {userId, status:'busy'}) to office room

User disconnects

- Server: User.update({status:'offline'})
- Broadcast: emit('presence', {userId, status:'offline'}) to office room

Storage: users collection → status field

11.8 All Socket.IO Events

Event	Direction	Payload	Purpose
dm_send	Client → Server	{ toId, content }	Send a DM
dm_message	Server → Client	{ _id, from, to, content, createdAt }	Deliver a DM

knock	Client → Server	{ toId, roomId }	Initiate a call
incoming_call	Server → Client	{ fromId, fromName, roomId }	Incoming call alert
call_accept	Client → Server	{ toId, roomId }	Accept a call
call_accepted	Server → Client	{ roomId }	Call was accepted
call_decline	Client → Server	{ toId }	Decline a call
call_declined	Server → Client	{ byId }	Call was declined
call_invite	Client → Server	{ toId, roomId }	Invite to ongoing room
set_status	Client → Server	status string	Update user status
presence	Server → Client	{ userId, status }	Status change broadcast
notification	Server → Client	notification object	New notification
recording:started	Both	{ roomName }	Recording started in room
recording:stopped	Both	{ roomName }	Recording stopped in room

12. Data Storage — Where Is Everything Saved?

All application data is stored in **MongoDB**. In development, an embedded MongoDB instance (`mongodb-memory-server`) runs automatically — no external database needed. Data persists across restarts in `apps/server/.data/mongodb/` .

12.1 MongoDB Collections

Collection	What It Stores	Key Fields
users	User accounts & credentials	name, email, password (bcrypt hashed), status, role, officeId, refreshTokens[]
offices	Office/workspace info	name, ownerId, members[], inviteCodes[] (code + expiry)
messages	Direct messages between users	from (userId), to (userId), content, read, createdAt
notifications	Knocks, invites, system alerts	userId, type, message, read, createdAt
recordings	Recording metadata (not the file itself)	roomName, egressId, filename, status, size, duration, recordedBy

12.2 File System Storage

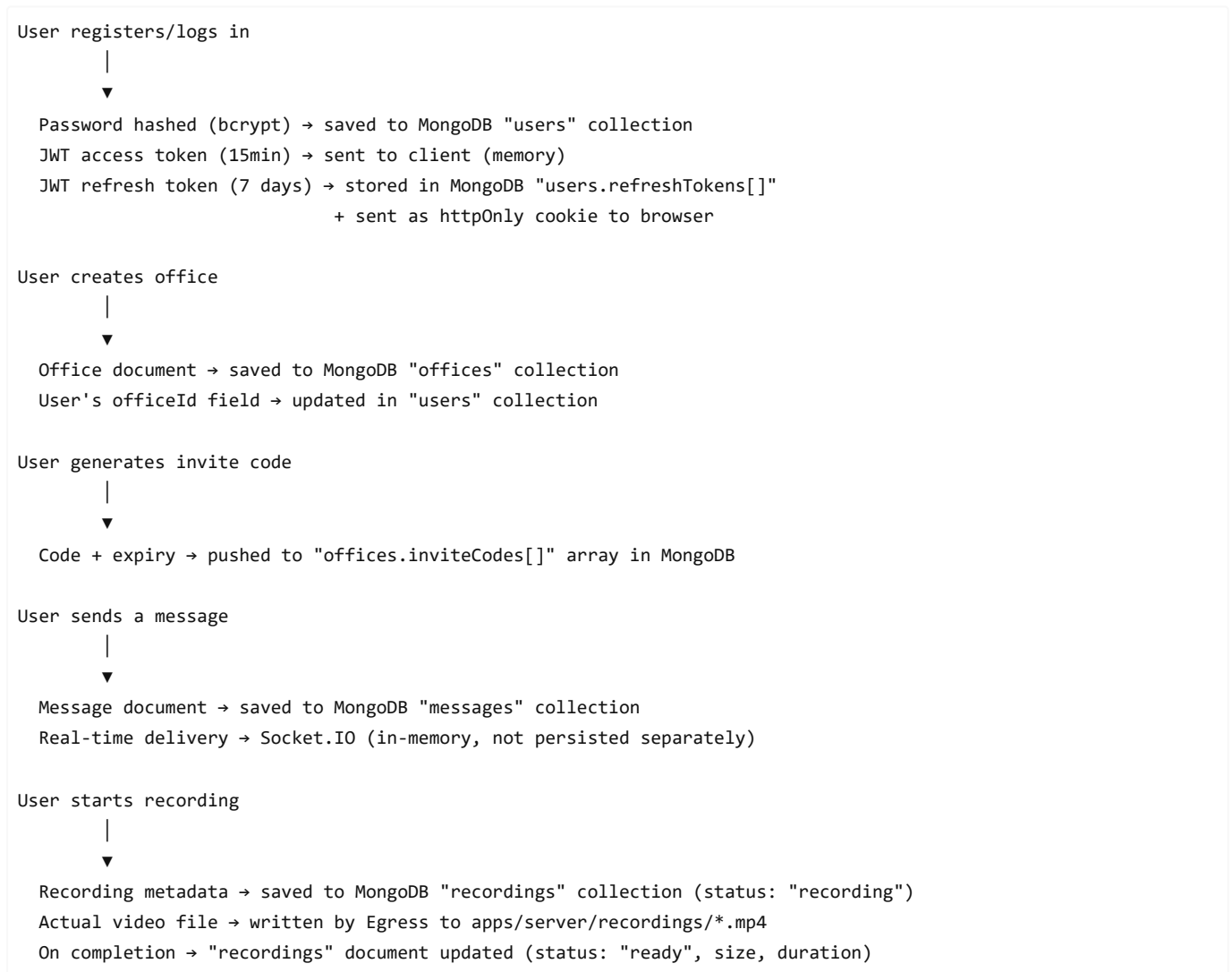
Data	Location	Format
Recording video files	<code>apps/server/recordings/</code>	MP4 (H.264 + AAC)
Embedded MongoDB data	<code>apps/server/.data/mongodb/</code>	WiredTiger engine files
Embedded MongoDB URI	<code>apps/server/.data/mongodb-uri</code>	Text file (connection string)

12.3 Where Specific Data Lives

Question	Answer
Where are user passwords stored?	In the <code>users</code> collection, hashed with bcrypt (never stored in plain text)

Where are login sessions stored?	Refresh tokens are stored as an array inside each <code>users</code> document (max 5 per user). Access tokens are stateless JWTs — not stored server-side.
Where are invite codes stored?	Embedded inside the <code>offices</code> document as an array of <code>{ code, expiresAt }</code> objects
Where are office members tracked?	The <code>offices</code> collection has a <code>members[]</code> array of user IDs. Each <code>users</code> document also has an <code>officeId</code> field.
Where are chat messages saved?	In the <code>messages</code> collection — one document per message with <code>from</code> , <code>to</code> , <code>content</code> , and timestamps
Where are recordings saved?	Metadata (name, status, duration) → <code>recordings</code> collection in MongoDB. Actual MP4 file → <code>apps/server/recordings/</code> on the filesystem.
Where are notifications saved?	In the <code>notifications</code> collection — one document per notification

12.4 Data Flow Diagram



12.5 Development vs Production

Aspect	Development	Production
--------	-------------	------------

MongoDB	Auto-embedded via <code>mongodb-memory-server</code> , data in <code>.data/mongodb/</code>	External MongoDB Atlas or self-hosted (set <code>MONGODB_URI</code>)
Recordings	Local filesystem <code>apps/server/recordings/</code>	AWS S3 bucket (configure <code>AWS_*</code> env vars)
Redis	Docker container on port 6379	Managed Redis (e.g., AWS ElastiCache)
LiveKit	Docker containers (local)	LiveKit Cloud or self-hosted cluster